

Assignment 2

Data processing

Ngoc Bao Tran Nguyen
Student ID: 25611551
2025 SPRING

94693 – Big Data Engineering
Master of Data Science and Innovation
University of Technology of Sydney



Table of Contents

1. Project overview	2
2. Data Ingestion and Preparation.....	2
3. Business Questions	4
4. Machine Learning (use sklearn)	8
5. Limitation	10
6. Conclusion	10
7. Reference	11

1. Project overview

This project applies Databricks Spark to the New York City TLC taxi dataset, which contains millions of trip records from yellow and green taxis. The work involves three main stages: (1) data ingestion, cleaning, and joining to prepare a reliable dataset; (2) SQL-based analysis to answer business questions about trip volumes, fares, and patterns; and (3) building and evaluating machine learning models to predict total trip amounts, with performance assessed using RMSE. The project demonstrates an end-to-end big data workflow from processing to predictive modeling.



2. Data Ingestion and Preparation

Explore the dataset and remove unrealistic trips

- **Trips finishing before the starting time**

During data cleaning, we identified trips where the recorded drop-off time occurred before the pick-up time, which is logically impossible. To detect these cases, we calculated trip duration in seconds using the difference between drop-off and pick-up timestamps, and flagged records with $\text{duration} \leq 0$. The result showed 68,318 green taxi trips and 1,108,986 yellow taxi trips with invalid or negative durations.

- **Trips where the pickup/dropoff datetime is outside of the range**

We used SQL to check the min/max pickup and drop-off datetimes and found unrealistic extremes (e.g., year 2081 and 2253). To fix this, we applied valid ranges: January 2009–December 2024 for yellow taxis and August 2013–December 2024 for green taxis. After filtering, we removed 2,015 yellow trips and 816 green trips with invalid dates.

- **Trips with negative speed**

We next checked for trips with negative or zero average speed, which are unrealistic. Average speed was calculated as:

$$\text{Speed (km/h)} = (\text{trip_distance} \times 3600) / \text{trip_duration_sec}$$

Any records with positive trip duration but average speed ≤ 0 were flagged as invalid. The results showed 1,300,007 green taxi trips and 6,699,144 yellow taxi trips with negative or zero speed values.

- **Trips with very high speed (look for NYC and outside of NYC speed limit)**

We checked for unrealistic speeds using NYC DOT limits: 50 mph (80 km/h) within the city and 65 mph (105 km/h) outside (e.g., near airports). After classifying trips with taxi zone data, we flagged 202,333 green and 782,768 yellow trips above the city limit, plus 14,907 green and 375,131 yellow trips above the outside limit.

- **Trips that are travelling too short or too long (duration wise)**

To detect unrealistic trip durations, we applied thresholds of at least 1 minute and at most 10 hours. Trips shorter than 1 minute are unlikely to represent valid journeys, while those longer than 10 hours are implausible for NYC taxi services and likely due to recording errors.

Using these rules, we flagged 1,713,485 green taxi trips and 8,562,722 yellow taxi trips as duration outliers.

- **Trips that are travelling too short or too long (distance wise)**

We applied thresholds to remove unrealistic trips based on distance. Trips shorter than 0.1 miles (≈ 160 meters) are likely invalid, as they may represent GPS errors or cancelled rides. Conversely, trips longer than 100 miles are implausible for typical NYC taxi operations, as they extend far beyond the city area.

Using these thresholds, we identified 403,381 green taxi trips and 1,742,825 yellow taxi trips as distance outliers.

- **Invalid passengers**

Passenger count is an important feature since it reflects trip occupancy and can affect fare calculations. We checked for invalid records where `passenger_count` ≤ 0 , which are not realistic for a taxi trip. The results showed 60,805 green taxi trips and 5,627,509 yellow taxi trips with invalid passenger counts.

- **Trips with total amount ≤ 0**

The `total_amount` field represents the fare charged for each trip, which in real-world operations should normally be greater than zero. Although in practice customers may occasionally use promotions or vouchers for free rides, our dataset does not contain variables to capture such cases. Therefore, any trip with `total_amount` ≤ 0 is considered invalid, likely caused by recording errors, test entries, or incomplete transactions.

The results showed 367,302 green taxi trips and 1,995,629 yellow taxi trips with non-positive total amounts.

- **Trips out of RatecodeID**

The `RatecodeID` field indicates the type of fare. Our check identified 1,885,146 green taxi trips and 9,560,380 yellow taxi trips with invalid `RatecodeID` values.

We then gathered all invalid subsets for both green and yellow taxis and built distinct invalid key sets. Using an anti-join by combining pickup/dropoff datetime, locations, distance, passenger count, and fare fields, we removed invalid trips from the base datasets.

- **Duplicate rows**

To ensure dataset uniqueness, we generated a trip signature combining pickup/dropoff time, locations, distance, passenger count, fare, and total amount. Duplicate trips were identified by ranking rows within each group and keeping only the first valid occurrence.

The results showed 1 duplicate trip in the green taxi dataset and 621 duplicate trips in the yellow taxi dataset.

Count the total number of rows for both green and yellow taxis

After applying all cleaning steps — including removing trips with invalid times, speeds, durations, distances, passenger counts, fares, rate codes, and duplicates — we obtained the final cleaned datasets for analysis.

- Green taxi: 78,896,049 valid trips remained (94.50%).
- Yellow taxi: 879,379,013 valid trips remained (96.85%).



Figure 1: Number of rows for green (left) and yellow taxi (right)

Combine and save the final cleaned and joined dataset as a table

After cleaning, green and yellow taxi datasets were aligned, tagged with a taxi_color field, and merged into one unified dataset. Pickup and dropoff IDs were mapped to boroughs and zones using the NYC taxi lookup table. The final dataset, saved as final_trips in Databricks, contains 958,275,062 valid trips.

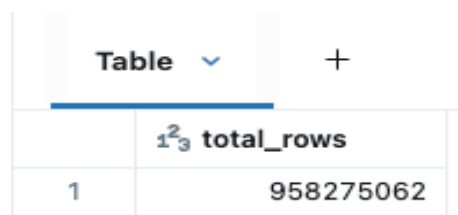


Figure 2: The number of rows of final table

3. Business Questions

For each year and month, we used SQL to check the total number of trips, day of week had the most trips, hour of the day had the most trips, the average amount paid per trip and the average amount paid per passenger. The table showed 135 results, below are the first 15 rows as a representative sample:

	1.0 year_month	1.0 total_trips	1.0 most_trips_day	1.0 most_trips_hour	1.2 avg_passengers	1.2 avg_amount_per_trip	1.2 avg_amount_per_passenger
1	2009-01	996	Thursday	0	1.67	18.57	15.34
2	2011-01	2	Monday	23	1	12.8	12.8
3	2011-02	1	Tuesday	0	1	15.8	15.8
4	2014-01	14425615	Friday	19	1.69	14.37	11.64
5	2014-02	13914006	Saturday	19	1.68	14.5	11.8
6	2014-03	16539773	Saturday	19	1.68	14.66	11.93
7	2014-04	15761536	Wednesday	19	1.68	14.95	12.14
8	2014-05	16020479	Friday	19	1.68	15.52	12.57
9	2014-06	14982745	Sunday	19	1.68	15.53	12.58
10	2014-07	14219362	Thursday	19	1.68	15.2	12.3
11	2014-08	13700500	Friday	19	1.68	16.13	13.15
12	2014-09	14556907	Tuesday	19	1.66	15.58	12.69
13	2014-10	15617746	Friday	19	1.66	15.58	12.71
14	2014-11	14666966	Saturday	19	1.66	15.37	12.51
15	2014-12	14563279	Wednesday	19	1.66	15.55	12.65

135 rows | 1m 23s runtime

Refreshed 4 days ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Figure 3: The first 15 rows as a representative sample for the results of each year and month

Monthly trips often exceeded 13–16 million in 2014. The busiest periods were typically Fridays and Saturdays, while the average passenger count stayed stable at 1 – 2 person per trip. Fares were also consistent, averaging \$14–16 per trip or \$11–13 per passenger.

	1.0 taxi_color	1.0 avg_duration_min	1.0 median_duration_min	1.0 min_duration_min	1.0 max_duration_min	1.2 avg_distance_km	1.2 median_distance_km
1	green	13.79	10.52	1.00	600.00	4.77	3.11
2	yellow	14.41	11.27	1.00	600.00	4.9	2.74

2 rows | 2m 50s runtime

Refreshed 4 day

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

	1.2 median_distance_km	1.2 min_distance_km	1.2 max_distance_km	1.2 avg_speed_kmh	1.2 median_speed_kmh	1.2 min_speed_kmh	1.2 max_speed_kmh
	3.11	0.16	160.89	20.39	18.54	0.02	104.48
	2.74	0.16	160.87	18.88	16.58	0.02	104.61

2 rows | 2m 50s runtime

Refreshed 4 days ago

It is stored as `_sqlidf` and can be used in other Python and SQL cells.

Figure 4: The result table of each taxi color

On average, yellow taxis had slightly longer trips (14.41 minutes, 4.9 km) compared to green taxis (13.79 minutes, 4.77 km). Median values show yellow trips were shorter in distance (2.74 km) than Green (3.11 km). Both taxi types had the same minimum duration (1 min) and minimum distance (0.16 km), while maximum duration reached 600 minutes and maximum distance exceeded 160 km. Average speeds were close (Green: 20.39 km/h, Yellow: 18.88 km/h), with maximum speeds above 100 km/h.

For each taxi color, each pair of pick up and drop off locations, each month, each day of week and each hour, we checked the total number of trips, the average distance, the average amount paid per trip and the total amount paid. The table showed over 10,000 results, below are the first 14 rows as a representative sample:

	taxi_color	pu_borough	do_borough	year_month	day_of_week	hour_of_day	total_trips	1.2 avg_distance_km	1.2 avg_amount_per_trip	1.2 total_amount_paid
1	green	Bronx	Bronx	2014-01	Friday	0	187	3.69	10.17	1900.98
2	green	Bronx	Bronx	2014-01	Friday	1	142	3.39	9.61	1365.1
3	green	Bronx	Bronx	2014-01	Friday	2	82	3.94	10.33	847.25
4	green	Bronx	Bronx	2014-01	Friday	3	63	3.41	9.72	612.06
5	green	Bronx	Bronx	2014-01	Friday	4	56	3.32	9.79	548
6	green	Bronx	Bronx	2014-01	Friday	5	68	3.67	9.81	667
7	green	Bronx	Bronx	2014-01	Friday	6	130	4	10.05	1306.1
8	green	Bronx	Bronx	2014-01	Friday	7	501	3.48	10.08	5048.6
9	green	Bronx	Bronx	2014-01	Friday	8	815	3.35	10.13	8254.29
10	green	Bronx	Bronx	2014-01	Friday	9	578	3.44	9.93	5741.19
11	green	Bronx	Bronx	2014-01	Friday	10	561	3.73	10.57	5929.32
12	green	Bronx	Bronx	2014-01	Friday	11	550	3.57	10.16	5587.14
13	green	Bronx	Bronx	2014-01	Friday	12	577	3.56	10.23	5904.86
14	green	Bronx	Bronx	2014-01	Friday	13	577	3.62	10.66	6151.1

10,000+ rows | Truncated data | 1m 28s runtime

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Figure 5: The first 14 rows as a representative sample

Trips are concentrated within the same boroughs, with Bronx-to-Bronx shown as an example, where demand steadily increases from early morning and peaks around 8–10 AM and again in the evening. Average trip distances remain short (around 3–4 km), reflecting mostly local travel within boroughs. Average fares per trip stay relatively stable at about \$9–11, while total fare revenue scales directly with trip volume. These findings show that while individual trip costs are consistent, borough-level and time-based demand strongly drive revenue.

	pu_borough	do_borough	1.2 revenue_2024	1.2 share_pct	1.2 cumulative_share_pct
1	Manhattan	Manhattan	636474595.18	61.12	61.12
2	Queens	Manhattan	171174682.75	16.44	77.56
3	Manhattan	Queens	68967529.2	6.62	84.18
4	Queens	Brooklyn	37614319.68	3.61	87.79
5	Manhattan	Brooklyn	31645764.43	3.04	90.83
6	Queens	Queens	31107670.68	2.99	93.82
7	Queens	Unknown	14713291.51	1.41	95.23
8	Manhattan	EWB	11934728.1	1.15	96.38
9	Manhattan	Unknown	6839433.24	0.66	97.04
10	Queens	Bronx	5755547.26	0.55	97.59

10 rows | 7.28s runtime

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Figure 6: Top 10 pickup - dropoff borough pairs in 2024

In 2024, taxi revenues were highly concentrated in a few key borough flows. Manhattan→Manhattan alone contributed 61.12%, followed by Queens→Manhattan (16.44%) and Manhattan→Queens (6.62%), together making up 84.18% of all revenue. Other flows such as Queens→Brooklyn and Manhattan→Brooklyn each contributed less than 4%. This highlights Manhattan’s central role in driving taxi demand and revenue across the city.

Table ▾		+
	.00 pct_trips_with_tips	
1	63.16	

Figure 7: The percentage of trips where drivers received tips

In total, 63.16% of all **taxi trips included a tip**, indicating that nearly two-thirds of riders provide additional payment beyond the base fare, which represents an important supplemental income source for drivers.

Table ▾		+
	.00 pct_tips_ge_15_among_tipped	
1	0.82	

Figure 8: The percentage of trips where driver received tips of at least \$15

Among all trips where drivers received a tip, only 0.82% of them were \$15 or higher. This shows that large tips are rare, with the majority of tips being relatively small amounts.

Table ▾		+		
	A _C duration_bin	I ₃ trips	1.2 avg_speed_kmh	1.2 avg_km_per_dollar
1	Under 5 mins	133495252	19.77	0.163
2	5–10 mins	284553771	17.2	0.21
3	10–20 mins	339131309	17.89	0.257
4	20–30 mins	121957969	21.33	0.305
5	30–60 mins	70834208	25.8	0.374
6	≥ 60 mins	8302553	22.47	0.629

Figure 9: Classify each trip into bins of durations

Most trips fall in the 5–20 minute range, with over 600 million rides. Short trips (under 5 minutes) are also very common. Average speed increases with longer trips, peaking at **25.8 km/h** for the 30–60 minute range. Efficiency (km per dollar) is lowest for very short trips (0.163) and highest for long trips over 60 minutes (0.629), showing that longer rides provide better distance value per dollar spent.

Table ▾		+				
	A _C duration_bin	I ₃ trips	.00 avg_duration_min	1.2 avg_dollars_per_hour	1.2 median_dollars_per_hour	.00 trip_share_pct
1	Under 5 mins	133495252	3.53	133.88	116.13	13.93
2	5–10 mins	284553771	7.42	84.18	78.97	29.69
3	10–20 mins	339131309	14.18	69.14	64.17	35.39
4	20–30 mins	121957969	24.11	68.4	59.5	12.73
5	30–60 mins	70834208	39.29	72.93	66.39	7.39
6	≥ 60 mins	8302553	78.19	58.73	55.95	0.87

Figure 10: Average of duration, average of revenue per hour, trips share percentage of each bin

Analysis of trip durations shows that while the shortest trips (under 5 minutes) yield the highest dollars per hour (\$133.88 on average), they represent only 13.9% of all trips and may be inefficient due to extra time spent waiting or repositioning between rides. In contrast, the 5–10 minute bin strikes the best balance: it captures nearly 30% of total trips, offers strong earning potential at around \$84 per hour, and ensures a steady stream of passengers. For this reason, we suggest drivers prioritize 5–10 minute trips, as they provide both sustainable demand and reliable income efficiency.

4. Machine Learning (use sklearn)

For the prediction task, we selected data from January 2023 to September 2024 as the training and validation period, and reserved October to December 2024 as the prediction target. This ensures the model is trained on recent patterns while still having a clear future window for testing, which aligns with real-world forecasting scenarios.

Before modeling, we examined the top 10 trips by total amount in 2023–2024. The results revealed some extreme outliers: trips with very short distances but unusually high fares. These anomalies can distort the model and reduce predictive accuracy. To address this, we applied clipping rules to filter unrealistic records, ensuring the dataset better reflects typical taxi trip behavior.

	taxi_color	pu_borough	do_borough	pickup_ts	dropoff_ts	1.2 total_amount	1.2 distance_miles	1.2 distance_km
1	ow	Manhattan	Manhattan	2023-06-12T13:33:06.000+00:00	2023-06-12T13:44:13.000+00:00	386987.63	1.5	2.41
2	ow	Manhattan	Manhattan	2024-11-01T16:15:08.000+00:00	2024-11-01T16:30:32.000+00:00	335550.94	2	3.22
3	ow	Manhattan	Queens	2023-09-02T15:15:39.000+00:00	2023-09-02T16:12:39.000+00:00	187513.9	21.3	34.28
4	ow	Manhattan	Manhattan	2023-09-05T10:16:13.000+00:00	2023-09-05T10:20:56.000+00:00	143167.45	0.7	1.13
5	ow	Manhattan	Unknown	2023-05-16T10:12:28.000+00:00	2023-05-16T14:47:02.000+00:00	6304.9	40.81	65.68
6	ow	Manhattan	Manhattan	2024-11-15T12:13:52.000+00:00	2024-11-15T12:22:06.000+00:00	5910.84	0.1	0.16
7	ow	Manhattan	Manhattan	2024-12-04T14:24:18.000+00:00	2024-12-04T15:07:17.000+00:00	3037.1	2	3.22
8	ow	Queens	Unknown	2023-04-06T14:08:51.000+00:00	2023-04-06T19:11:17.000+00:00	2451	12.58	20.25
9	ow	Manhattan	Brooklyn	2023-12-20T18:49:49.000+00:00	2023-12-20T19:16:51.000+00:00	2372.79	6.7	10.78
10	ow	Manhattan	Queens	2024-04-06T16:33:28.000+00:00	2024-04-06T17:01:26.000+00:00	1737.18	5	8.05

Figure 11: Top 10 trips by total amount in 2023 - 2024

We clipped at the 1st and 99th percentiles because percentile-based thresholds are more robust to skewed distributions and extreme outliers than mean–standard deviation methods.

We used variables such as trip_distance, passenger_count, month, hour, dow, taxi_color, payment_type, RatecodeID, PULocationID, DOLocationID to train and predict total_amount. Therefore, we did not use variables related to amount to avoid dataleakage.

We used variables such as trip_distance, passenger_count, month, hour, dow, taxi_color, payment_type, RatecodeID, PULocationID, DOLocationID to train and predict total_amount. Therefore, we did not use variables related to amount to avoid dataleakage.

To prepare for prediction, we adopted a time-based split. Data from January 2023 to September 2024 was used as the training set (62,406,599 rows) to capture recent patterns and seasonality. The

test set consists of data from October to December 2024 (9,744,182 rows), representing the future period we aim to predict.

Table ▾ +			Table ▾ +			
	i_3^2 year	i_3^2 count		i_3^2 year	i_3^2 month	i_3^2 count
1	2023	36114932	1	2024	10	3336629
2	2024	26291667	2	2024	11	3177707
			3	2024	12	3229846

Figure 12: The number of rows of train set (left) and test set (right)

After defining the train and test sets, we converted the dataset to Pandas for model training. To optimize memory usage, the conversion was done column by column. Importantly, although variables such as `payment_type`, `RatecodeID`, `PULocationID`, and `DOLocationID` are stored as numbers, they are categorical by nature. Therefore, we explicitly cast them to category dtype in Pandas. This reduces memory consumption and ensures the model interprets these fields correctly as categorical rather than continuous numerical variables. Finally, we converted variables into their correct data type, saved train set and test set in Parquet.

Baseline Model:

We first built a baseline model to establish a reference point. Using the clipped dataset (Jan 2023–Sept 2024), the model predicts fares as the average total amount per group, defined by taxi color, pickup and drop-off locations, month, day of week, and hour. Predictions for Oct–Dec 2024 were then compared with actual values. The baseline achieved an RMSE of **10.31**, this error is relatively high, highlighting the need for more advanced machine learning models.

Before training, we log-transformed `trip_distance` to reduce skewness and one-hot encoded `taxi_color` to handle it as a categorical feature. These steps improved data representation for modeling.

ElasticNet Model:

We scaled numeric features and applied one-hot encoding to categorical codes (e.g., `payment_type`, `RatecodeID`, `PULocationID`) so the model treated them correctly as categories. To avoid memory issues, we trained ElasticNet on 10% of the data (6,240,660 rows), which was still representative.

ElasticNet was chosen for its balance of L1 and L2 regularization, handling correlated features well. With $\alpha=0.1$ and `l1_ratio=0.5`, the model achieved a CV RMSE of 6.29, train RMSE of 6.29, and test RMSE of 6.76—substantially improving over the baseline RMSE of 10.31.

XGBoost Regressor Model:

We used XGBoost Regressor, which handles categorical integer features directly (e.g., `payment_type`, `RatecodeID`, `PULocationID`), reducing memory usage compared to one-hot encoding. The model was trained on a 30% sample (around 18M rows) and achieved strong results:

Train RMSE 4.71, CV RMSE 4.72, and Test RMSE 5.4057 — outperforming both the baseline (10.31) and ElasticNet (6.76).

To further optimize performance, we manually tuned key hyperparameters in stages (`max_depth/min_child_weight`, `subsample/colsample_bytree`, and `learning_rate` with `n_estimators`). The final tuned model reached RMSE 5.40, confirming XGBoost’s ability to capture complex non-linear patterns and making it the best model in this project. However, it took us half an hour to tune and train this model.



5. Limitation

This assignment faced several limitations. A major challenge was the extremely large dataset, which frequently caused out-of-memory errors during preprocessing and model training. To handle this, we had to repeatedly sample subsets of the data (e.g., 10% for ElasticNet and 30% for XGBoost), which may reduce the representativeness of results.

Working in Databricks also required constant resetting and re-running of cells to free memory, adding overhead to experimentation. Additionally, converting Spark DataFrames to Pandas was necessary for scikit-learn models but further strained memory and limited the scale of models we could train.

For categorical variables like `payment_type`, `RatecodeID`, and location IDs, one-hot encoding was required for ElasticNet, which increased memory usage, while XGBoost handled raw integer categories directly—this inconsistency reflects trade-offs made due to system constraints. Lastly, clipping the dataset helped remove unrealistic outliers but risked excluding rare yet valuable cases, while the absence of external data (e.g., traffic, weather, policy changes) limited the explanatory power of the models.



6. Conclusion

In this assignment, we developed machine learning models to predict taxi fares using data from January 2023 to September 2024, with October–December 2024 as the test set. After cleaning, clipping outliers, and defining a baseline, we compared ElasticNet and XGBoost. ElasticNet provided a simple linear benchmark, while XGBoost delivered significantly lower RMSE, showing its ability to capture complex relationships across features.

For future work, we recommend moving to Spark MLlib or other distributed frameworks to avoid Pandas memory limits, and leveraging GPU or cloud clusters to reduce training reset. Automated hyperparameter tuning with distributed search tools would further improve model performance. These steps would create a more scalable and efficient workflow for future large-scale taxi fare prediction tasks.



7. Reference

Scikit-learn documentation — ElasticNet: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.ElasticNet.html

Databricks Documentation – Scikit-learn on Databricks: <https://docs.databricks.com/en/machine-learning/train-model/scikit-learn.html>

Databricks Documentation – Pandas on Spark (handling large data / sampling): <https://docs.databricks.com/en/pandas/index.html>

Fitting Data in Chunks vs. Fitting All at Once in scikit-learn: <https://www.geeksforgeeks.org/machine-learning/fitting-data-in-chunks-vs-fitting-all-at-once-in-scikit-learn/>